For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Microservices Curriculum

- HLD - 18 - Monolith vs Microservices vs SOA
- HLD - 19 - Microservices - Communication, Observability, Resilience
- HLD - 20 - Microservices - Data Management, Consistency & Distributed Trans…

Reference resources

1. Microservices.io
2. Martin Fowler

Caution: Do NOT read these resources end-to-end.
Just treat them as "dictionaries". Whenever you encounter a term (related to software patterns) in real life and don't know what it means, look it up in these websites.

" Practicality beats Purity "

The "other" type of **Scale**

So far, we've seen that "Scale" means
1. amount of data stored
2. number of requests per second (read/write, avg/peak)

This type of scale determines our database choices, how we cache, our performance numbers, etc.

There's yet another form of Scale, that appears when the product complexity increases
1. number of products/features
2. code complexity & size
3. team size — number of developers in your organization

This form of scale influences our architectural & design choices – how we structure our code, how we structure our teams, etc.

Just like building a service is different for `1,000 vs 1 billion` users, building a product is very different with `10 vs 1000` developers.

For example, consider a large scale E-Commerce website like Amazon/Flipkart/Myntra/..

E-Commerce Zoo

E-commerce websites have a ton of requirements, that can be categorized into the following major modules (non-exhaustive)

Customer

- **User Management:** register, login, password recovery, OTPs, social auth
- **User Preferences:** address management, payment methods
- **Search:** sort, filters, typeahead
- **Product Listing:** product details, pricing, discounts, customizations
- **Cart Management:** add/remove, update quantity, concurrency
- **Wishlist:** CRUD, price drop alerts
- **Order Management:** CRUD, order history, cancellations, refunds, returns
- **Payments:** gateways, refunds, invoicing
- **Shipping:** tracking, logistics, address verification, delivery updates, returns
- **Reviews & Ratings:** reviews, aggregate ratings, abuse reports
- **Customer Support:** ticketing, live chat, FAQs

Vendor

- **Vendor Management:** auth flow, verification, profile management
- **Inventory Management:** products, categories, update listings, stock tracking, updates, alerts, replenishment
- **Order Processing:** accept/reject orders, shipping labels
- **Financial:** earnings dashboard, product analytics, traffic insights, customer sentiment
- **Promotion management:** discounts, coupon codes, ad campaigns, sponsored listings
- **Support:** ticketing, resolutions

Platform

- **Notifications:** email, SMS, push notifications, alerts
- **Recommendation system:** compare products, recommendations, similar products
- **Security & Compliance:** fraud detection, ..

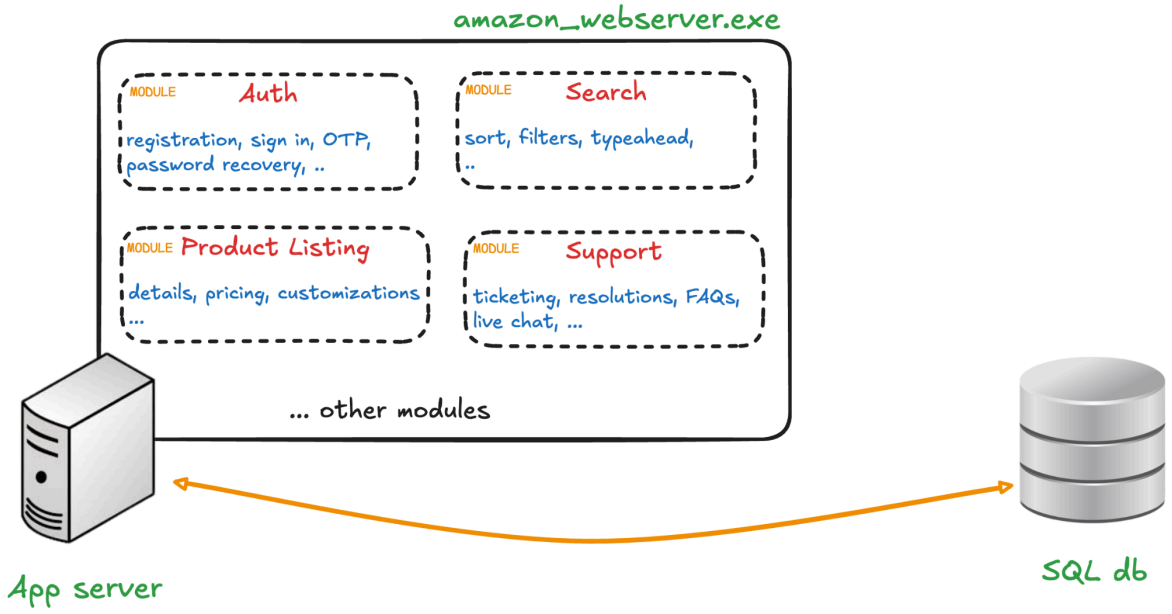
Admin

- **Analytics & Reporting:** sales metrics, user insights, inventory analytics
- **User Management:** user control, product moderation, order oversight
- …

The question now is, given this enormous list of requirements, how should we “architect” our code?

Monolithic Architecture

Unified model where all components *(interface, business logic, data access, ...)* are packaged & deployed as a single unit. This typically means one large codebase, and one database serving the entire application.



Monoliths emphasize ACID transactions within a single SQL database, ensuring strong consistency in one place.

Does monolith mean a single module?

No. A monolith can have multiple internal modules, but they run in the same process.
Why should we have modules? To manage code complexity — it is just a good coding practice to “decouple” your code into various functions/classes/modules/namespaces (Single Responsibility Principle)

Does monolith mean a single process?

All the modules run in the same process. But you can totally run multiple instances of the executable side-by-side in multiple processes (each process will be running all the modules)

Does monolith mean Vertical Scaling?

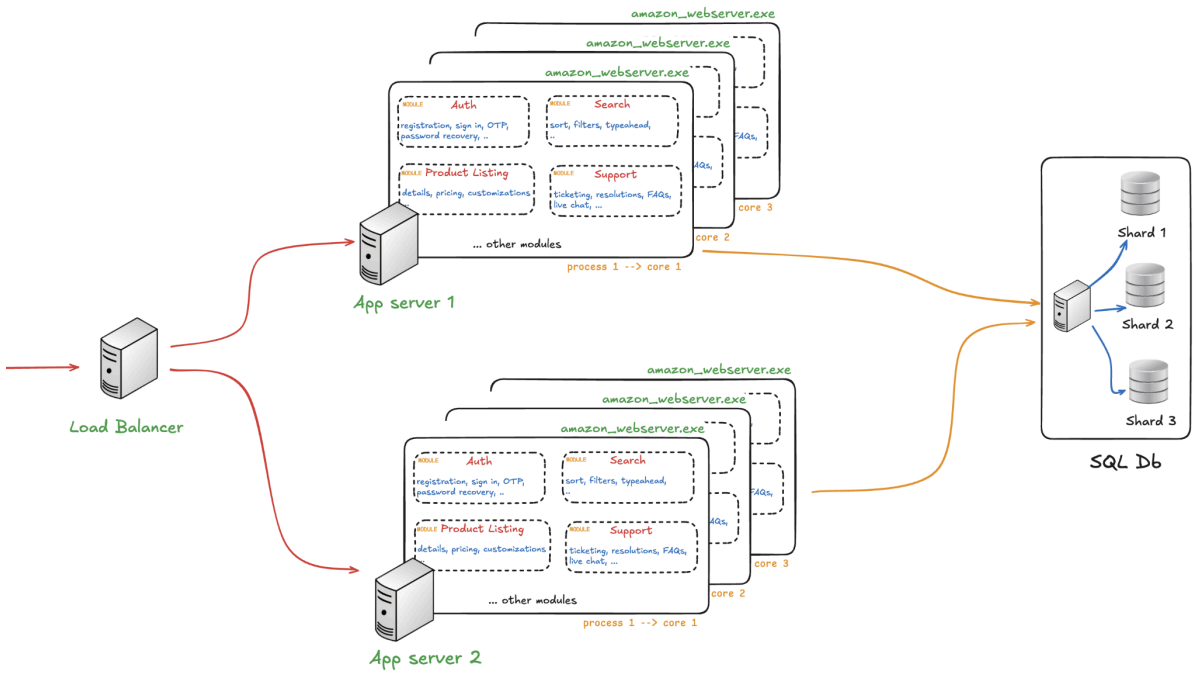
No. You can Horizontally Scale a monolith — just install the same executable on multiple servers (you will also need a Load Balancer)

Does monolith mean unsharded database?

No. Database can still be sharded & replicated, since it is separate from the app-servers

Does monolith mean SQL is must?

No. You can have other databases as well, but SQL is typically the de-facto.



Pros

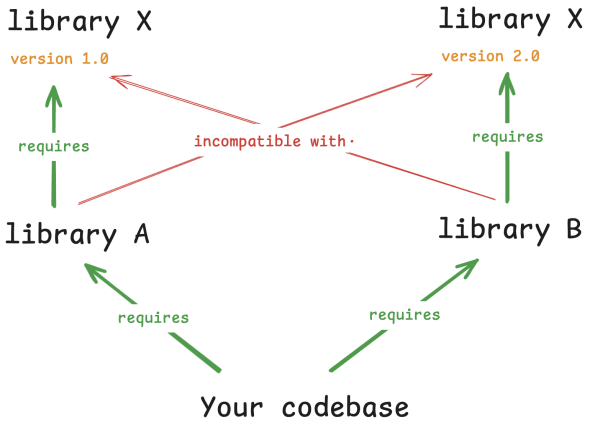
All these pros will apply at low scale *(small codebase, small team, few features & products)*

- **Straightforward to Develop & Deploy:** you run one `.war` or `.exe` and everything is up
- **Homogeneous webservers:** all the webservers run the exact same code
- **Predictable Performance:** all function calls are in-process
    - unpredictable performance delays due to network congestion will not happen
- **Simpler Debugging:** all function calls are in-process. You can step-through your code (just attach a debugger)
- **Simpler Logging:** the request life-cycle takes place completely within one server & one process. You just need to look at a single server's logs to understand what happened to the request
- **Easier Cross-Cutting concerns:** security, exception handling, monitoring, environment variables, connection pooling, etc
- **Avoids Network Latency:** everything is local – just make function calls
- **Easy transactions:** locks (mutex/semaphores) are easy when all modules are part of the same process
- **Expertise buildup:** your team develops expertise in the used tech-stack, and most code problems become easy to solve

Cons

As the complexity grows a Monolith can become “a large unit with several services” mashed together leading to **Tight Coupling**

- **Slow startup times:** the giant codebase needs to setup hundreds of dependencies before it can start
- **Merge Conflicts:** because all devs are working on the same codebase
- **Longer Release Cycles:** A change in one part (say the order module) requires rebuilding and redeploying the whole application
- **Coupled Scaling:** scaling must happen for the entire app even if only one feature needs more capacity
    - for example, the search service receives 1000x more requests than the support service
    - but if we want to scale out the search module, all the other modules will scale along with it
- **Frequent Failures:** a bug in any module (e.g., an out-of-memory in the reporting component) could crash the entire application since it's one process.
- **Technology Lock-in:** you’re stuck with a single tech stack (programming language, framework, libraries)
    - NLP/ML – wish to use Python, but can’t because you’re stuck with Ruby on Rails
    - Video processing – wish to use C++/Rust, but you can’t, because you’re stuck with Ruby on Rails
- **Slower Developer Onboarding:** any new developer must understand the massive codebase before making meaningful contributions
- **Dependency Hell:** due to a large number of features requiring a large number of dependencies, the changes of unresolvable dependency conflicts grows exponentially



- **Slower experimentation:** you can’t experiment quickly & break things, because your entire business depends on the single codebase
- **Slower upgrades:** can’t upgrade dependencies easily, because a lot of code would need to be updated
- **Mistakes compound:** mistakes will happen eventually, but fixing them will be expensive – so they will be left in

Overall, your codebase becomes a " big ball of mud "

When to Prefer

For small teams and projects with limited scope, a well-structured monolith can be easier to manage than a distributed system.

> " In the 2000s, **Amazon** started as a monolith (the "obidos" system). It was fine at first, but as Amazon's business exploded, the monolith became a bottleneck – small changes took too long to deploy, and one slow component (like the recommendations engine) could slow the whole site. This pain eventually pushed Amazon to break the monolith into microservices in the mid-2000s. "

Some (smaller) companies today intentionally stick with monoliths for speed.

If monoliths become so bad eventually, why start with one at all?

If you're building a new company from scratch in 2025, should you always start with microservices, because you know that the monolith will not scale?
NO!

- early on, you often don't know clear service boundaries
- monolith lets you get a product out quickly without the overhead of orchestrating microservices (you won't have to deal with complicated devops)

Starting with microservices too early can lead to trouble. Monolith-first strategy allows you to validate the product and find natural boundaries before splitting.
If and when the monolith truly becomes a bottleneck, you'll have a working product and better understanding of how to slice it.

YAGNI (You ain't gonna need it!) & KISS (Keep it simple, stupid) – don't introduce complexity (like a distributed system) before it's necessary

" almost all successful microservice stories have started with a monolith that got too big and was broken up "

— Martin Fowler

Can we make a monolith modular to delay or avoid microservices?

You can extend the life of a monolith by following good architecture practices like modular layering, clean interfaces, and bounded contexts, clearly separated modules, DDD (Domain-Driven Design).

Modular monolith: codebase is one deployable, but strongly split into internal services or components with enforced boundaries (possibly even separate schemas). This provides some benefits of microservices (team autonomy over modules) without introducing distributed complexity (at least up to a point).

Monolith → Microservices Difficult!
Monolith → Modular Monolith → Microservices Better!

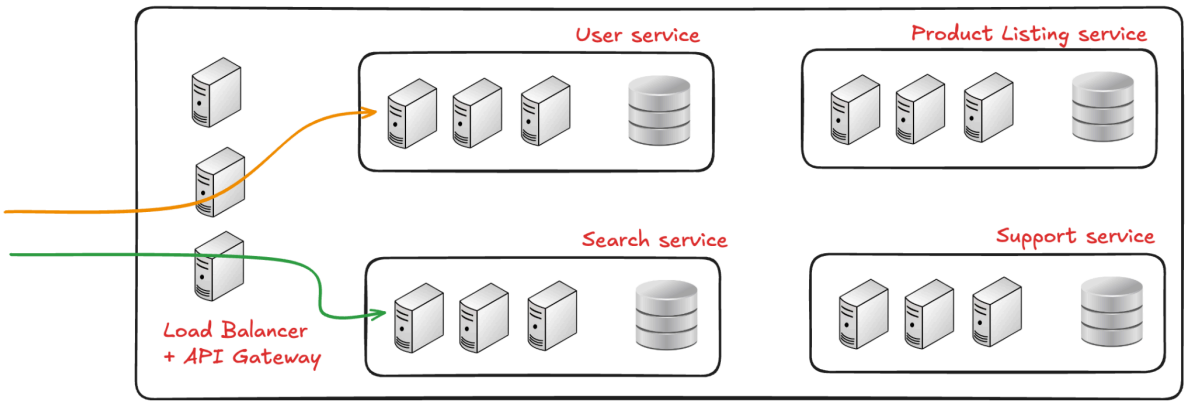
How will you know that the monolith is now no longer sufficient?

- your code changes will take longer
- compiled & startup times are very high
- very difficult to understand the code – onboarding new devs has become difficult
- performance slowdown is not always there, but development slowdown will start happening
    - shipping new features & fixes takes very long

Microservices Architecture

Collection of small services, each running in its own process and communicating via lightweight mechanisms *(often messaging queues, sometimes REST, Remote Procedure Calls (RPC))*.

Each microservice implements a single business capability and can be developed & deployed independently. Each service manages its own data, has its own team, and can choose the tech stack best suited to the service — decentralization.



Does each microservice mandatory have to have its own database?

"Purity" – yes. If you need to share data, use patterns like CQRS
"Practicality" — not necessarily

Pros

- **Service Autonomy:** each microservice can be updated on its own schedule — deployments whenever you want, without affecting other services
- **Loose Coupling for Communication:** via APIs *(more on this in the next class)*
    - congestion in one service will not cascade to other services
- **Bounded Contexts:** each service owns a specific business context, aligning with DDD principles
- **Isolation:** each service runs in isolation, one service's internal changes don't affect others as long as the API contract is stable
    - If the Recommendation Service has a memory leak and crashes, it doesn't take down the ordering system – it's a separate service
    - heads up - failures can still cascade despite isolation – to properly prevent them, we will need to use some additional patterns like Circuit Breaker *(next class)*
- **Single Responsibility Principle** at the service level – each does one thing well
- **Lightweight Communication:** often synchronous calls like REST/gRPC or asynchronous messaging
- **Database per service:** each microservice often has its own database
    - avoids shared schema coupling
    - but means cross-service data dependencies & consistency issues must be handled via other means (CQRS, distributed transactions)
- **Flexible Scaling:** you can scale hot services without over-provisioning the entire app
    - you can scale out the search service independent of the support service
- **Faster & Frequent Deployments** for different parts of the app – teams can deploy their service when ready, possibly many times a day, without a coordinated global release
    - Netflix, for instance, has *"more than a thousand microservices"* and deploys code thousands of times per day on parts of the system
- **Independent DevOps and CI/CD:** small teams owning services end-to-end (from development to deployment)
- **Independent Techstacks:** each team can use the tools (programming language, libraries) that best suit their usecase
    - ML people can use Python
    - Video people can use C++
    - Some UI team might use React
    - other UI team might use Svelte
    - **example: Scaler has a monolith – Ruby on Rails**
        - problem: when solving DSA problems, students raise "similar" help requests
        - because we run multiple batches, we've already seen a large majority (90%) of all types of help requests already

    - still, whenever a user raises a help request, they've wait for a TA to answer it
    - idea:
        - say you're solving Problem-3
        - we know what testcases exist, and give your submission, we know what testcases you code failed on
        - given your submission, fetch other submissions in the past that failed on the same testcases
        - and if someone in the past raised a help request (which was resolved with high rating), then just show this response to the current user
    - all the devs at Scaler were already busy
    - a new intern wanted to work on a ML task
    - we just asked the intern to duplicate the submissions table into a separate db
    - work on a microservice
    - Python
    - implemented this solution in 2 months
    - Help request resolution times decreased by 80%

Most of the "cons" that Monoliths have at scale are resolved by microservices

Cons

- **Added Operational Complexity:** instead of one application to manage, you now have dozens or hundreds of services – each needs deployment, monitoring, logging, observability
- **Additional Infrastructure:** API gateways, Observability, Message Queues, … (both server cost, and devops cost)
- **Network Overhead:** architecture is distributed, meaning network calls introduce latency and points of failure
- **Need for Failure handling:** need robust inter-service communication mechanisms to handle failures gracefully with retries, timeouts, etc.
- **Difficult Testing & debugging:** testing becomes harder – instead of one integrated codebase, you must test interactions across services
- **Data consistency** is more difficult because data is spread across many databases (no simple ACID across them).
    - deep dive in the 3rd microservice class

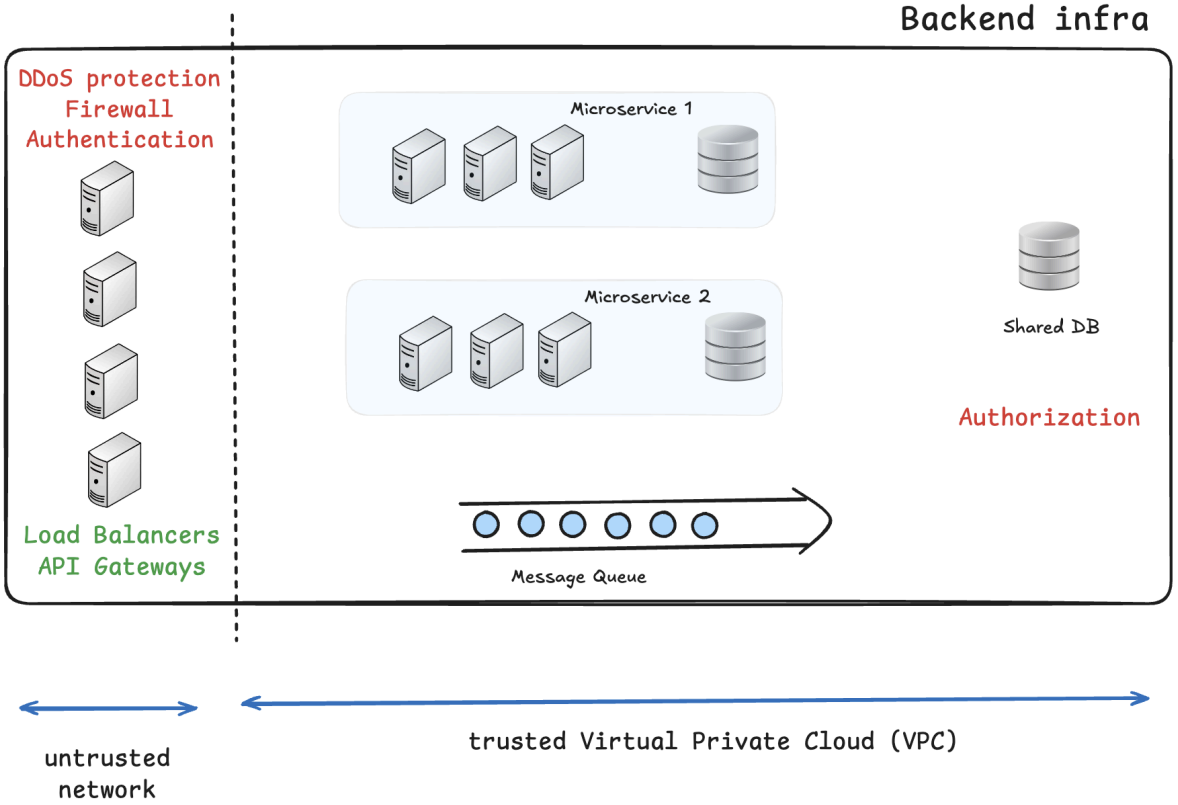
Microservice Premium: an upfront cost in complexity that only pays off for sufficiently large & complex systems.

" Microservices trade-off simpler centralized management for scalability, agility, and autonomy at scale "

“ Netflix has pioneered the Microservices architecture (they did it even before the term “Microservices” was coined!)

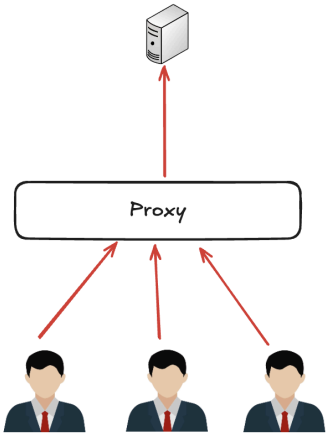
Around 2009, Netflix’s monolithic DVD-rental system struggled to keep up with new streaming demand. Netflix decided to migrate to AWS cloud and gradually broke the monolith into microservices. By 2015, Netflix had a fully microservices-based, cloud-native architecture, which contributed to their ability to scale for millions of users and stream reliably. They even won awards for this transformation. This architecture allowed independent evolution of features – for example, the “Recommendations” team could deploy a new algorithm without touching the “Streaming playback” service. ”

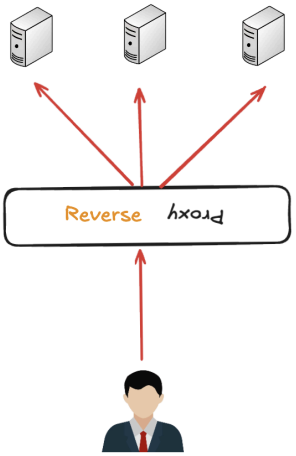
Common Components



- **Service:** collection of servers, caches, databases, load balancers, … other components working together towards a common business goal
    - A service will expose some API to the clients (end user or other services in the backend)
- **WebServer:** a computing machine (laptop/cloud virtual server/rack/raspberry pi) which executes the code for a service and exposes a web API

- **API Gateway:** forward the request to the appropriate service
  request for `/search?q=cat+food` should go to the search microservice and not to the user management service
  - API gateways are application layer – they need to understand the request (headers, decryption,..) to route the request to the correct service
- **Load Balancer:** evenly distribute the load across servers
  - don't need to understand the request..
  - they can work at Transport layer (frequently), sometimes, even at the Network layer
- **Reverse Proxy**





A proxy "pretends" to be you.
Instead of you sending request directly to the server
you send it via the proxy

From the server's perspective, the request is originating at the proxy
The server doesn't know anything about you.
You've effectly hidden yourself

My request is being forwarded
to the appropriate server

Note: APIGateway & LoadBalancer are both examples of Reverse Proxy. Others include caches, rate limiters, …
Both can sometimes be deployed in a single combined service. nginx, Kong, ..

- **Untrusted Layer:** All communication with the outside internet must pass through the untrusted layer. Provides a common point for security (firewall, DDoS protection) and authentication.
  Any SSL connections will terminate at the untrusted layer.

- **Trusted VPC:** Internal network for the servers & databases. Isolated from the outside network. We trust all components in the trusted VPC to not be malicious, hence there's no need for authentication at each service.
  Connections within the trusted VPC will be unencrypted (non SSL) & unauthenticated

- **Auth**
  - **Authentication:** am I who I claim to be?
    only happens at the untrusted layer

- **Authorization:** do I have to permission to access a particular resource? happens at every microservice!

When to use what

Start with a monolith, and use it until it hurts!

This applies to both companies that are just starting out, and even for large projects that are just starting out in massive companies.

[Monolith First – Martin Fowler]

- Almost all the successful microservice stories have started with a monolith that got too big and was broken up
- Almost all the cases where I've heard of a system that was built as a microservice system from scratch, it has ended up in serious trouble.

Initially, your startup will have

- small number of users `(< 10,000)` & small load
- few features & frequent requirements changes
- tight budgets & small team
- tight deadlines & lots of experimentation

You can not afford

- high infrastructure complexity & costs
- complex deployments
- hiring devops

So you should

- start with a monolith
- popular tech-stack
  - easier to hire
  - allows leveraging community solutions
  - for example, Scaler uses Ruby on Rails
    - Abhimanyu was the initial CTO
    - he had never worked with Ruby on Rails
    - Ruby on Rails was the most popular

Split into microservices only when clear pain points emerge.

Breaking a Monolith into Microservices

Imagine that you're the CTO of a 5 year old startup. You started with a monolith architecture, but over the years, the ***tech-debt*** has grown significantly. It has become difficult to add new features, things are breaking frequently, onboarding is slow..

As the CTO, you decide to migrate from monolith to microservices.
You tell your CEO that…

Proposal 1 — **Big Bang Rewrite**

- for the next 6 months the tech team will go into a refactoring mode
- no new features or bug fixes
- existing systems might break
- after 6 months, we will have a shiny new backend!

You're gonna get fired!

Proposal 2 — **Slow Strangle**

- for the next 2 weeks
- 2 out of the 50 developers will work on a special task
- break off search module from the existing monolith into a separate microservice
- infra costs will reduce by 20% because we will be able to scale independently
- no downtime for other services
- normal development & bug-fixes will continue as is
- revenue will likely increase by 5% because of faster search

You're getting a stock bonus!

Identifying the correct module

You can slice a monolith

- **Vertically** — by business capability (preferred)
    - meaning you extract an entire functional area (the UI, business logic, and data access) for that domain from end to end, instead of just one technical layer. For example, you might move everything related to "Order Processing" (including front-end components, back-end logic, and the database for orders) into its own service.
- ~~**Horizontally** — by technical layer (rarely recommended)~~
    - example, extract all database access layer into a service
    - that usually just creates a distributed monolith and isn't really microservices

To identify the correct module,

- identify something with few dependencies and significant business value
- look for “seams” in the monolith – places where you can cleanly separate components
- if there’s uneven load across components and some module needs to handle very high scale
    - example: search service sees 1000x more requests than the customer support service
- if the module strongly benefits from a different technology or runtime
    - say you have a heavy image processing component that would be best written in Rust or C++ for performance, while the rest of your system is a Ruby on Rails app
- if the module has special data storage needs (e.g., needs a graph database) that doesn’t fit the main SQL DB
- if the module fits the “two-pizza team” rule
    - *Jeff Besos: a microservice team should be small enough to be fed by two pizzas*

Once you’ve identified the module to separate out,

1. Build the new service for that piece
2. Route traffic or requests for that functionality to the new service
    - while the monolith possibly still handles everything else
3. Ensure the new service can work without breaking the user experience
    - often by syncing data or using anti-corruption layers
4. Incremental Steps & Monitoring
    - deploy a new service, redirect some traffic, and ensure all tests pass
        i. send 10% of requests to the new service, ensure results match, then 100%
    - feature flags, A/B tests
5. Iterate this process, each time aiming for “an atomic improvement to the overall architecture”
    - if two modules are heavily intertwined, you might first refactor to untangle their logic (within the monolith) creating a clear interface. Only then do you pull one out.

```
def _handleSearch_old(query: str):
    // earlier code

def _handleSearch_new(query: str):
    // API call to the new microservice

def handleSearch(query: str):
    p = random() // random number b/w (0, 1)
    if p < 0.1:
        return _handleSearch_new(query)
```

```
else:
    return _handleSearch_old(query)
```

This process may take multiple years!

- Amazon — famous two-pizza team - took 5+ years.
- Atlassian — "Project Vertigo" to split Jira/Confluence monoliths into microservices on cloud — took 2 years

Example: Rebuilding Netflix Video Processing Pipeline with Microservices

Conway's Law says your system design will mirror your org structure.

When to stay Monolith

- If your scale is modest and uniform, a monolith is simpler to manage
- Performance-critical integrated transactions may lean towards monolith (to avoid network hops)
    - *unless caching and other tricks can mitigate that in microservices.*
- If the domain is highly interwoven or you're in early stages where the boundaries are unclear, starting monolithic is wiser
    - Use monolith to iterate domain design quickly, then evolve to microservices when confident
- If uptime and fault tolerance of the module is more critical compared to other modules

You shouldn't start a new project with microservices, even if you're sure your application will be big. Build a well-structured monolith first and only evolve to microservices when necessary.

" Not every company is Google-scale "

Hybrid Architecture

You don't have to choose all-or-nothing

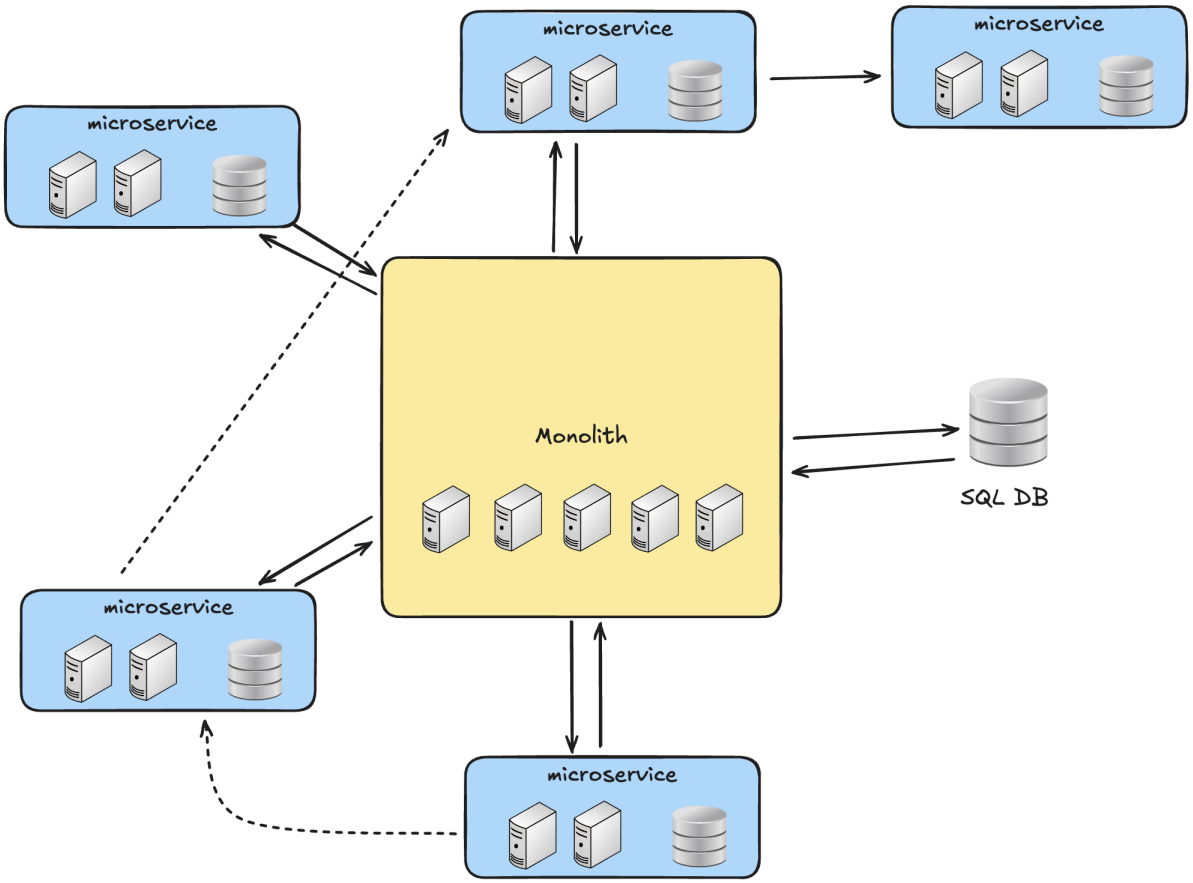
Modular Monolith incrementally extract microservices from an existing monolith. During this transition you have a hybrid: a core monolith still handling a lot, and a few new microservices alongside it. They might communicate via APIs. You effectively "strangle" the monolith by gradually moving functionality to services.

aka Strangler Pattern, Phased Migration, Incremental Refactoring, …

Phased Database Migration: Keep the monolith's DB as the system of record initially, but have new services use their own databases and gradually move data ownership. To keep the monolith and service in sync, techniques like change data capture or an **event bus** are used (the new service writes to its DB and also updates the old DB or vice versa) until the monolith's dependency on that data can be removed. This is a form of **transitional architecture** where for a time you maintain extra code to sync data between old and new systems.

aka Database Strangler.

The architecture spectrum is broad – it's not a binary monolith vs microservices, but a continuum.



Handling Cross-Cutting dependencies

Cross-Cutting dependency: common functionality (like a user authentication system, or a shared library of utilities).

For any cross-cutting dependency, we can either
- make it a separate microservice
- make it into a shared library

Examples

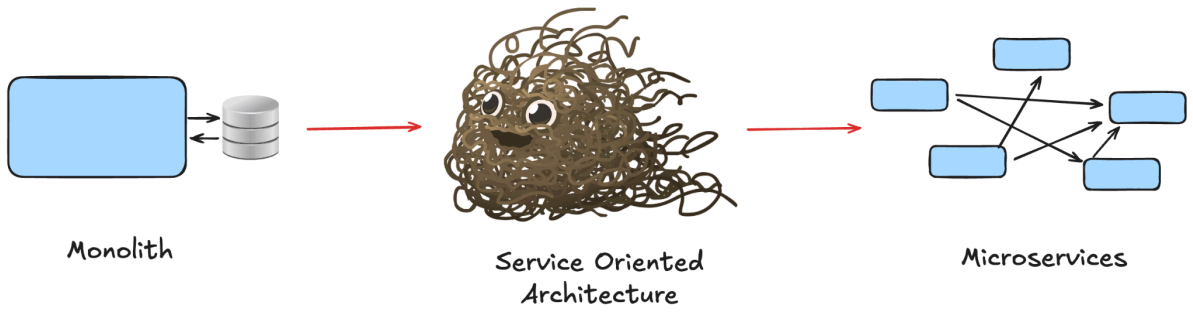
- **Security:** Typically handled via centralized authentication (e.g., an OAuth2 token service) and each service validating tokens (or mutual TLS between services for trust).
- **Transactions:** distributed transactions are tricky – patterns like Saga are used instead of 2PC.
- **UI consistency:** via a style guide and shared components, or *micro frontends* with coordination.
- **Utility functions:** become shared libraries
- **Logging, Error Handling:** *Service Mesh* or *Sidecar patterns*

Common Pitfalls in microservices adoption

- **Underestimating Operational Overhead:** automation, monitoring, containerization, CI/CD, logging, tracing
    - microservices will drown you in ops burden
- **Choosing Wrong Service Boundaries**
    - poor boundaries can lead to
        - tightly coupled services – where every little change requires touching many services
        - overly granular services – too many tiny services that could have been one, incurring needless network calls
    - using **Domain-Driven Design** helps by aligning services to business domains
    - **rule of thumb:** if two components change for the same reasons at the same time, they likely belong in the same service.
- **Ignoring Distributed Fallacies:**
    - common fallacies
        - the network is reliable
        - latency is zero
        - failures are always complete
        - data is always consistent
    - Always implement timeouts, retries.
    - Use patterns like circuit breakers.
    - Do distributed transactions if needed
- **Lack of Observability:**

- In a monolith, debugging is easy as – just look at one log file or attach a debugger to one process
- In microservices, a single user request might flow through many services
  - if something goes wrong, you need a way to trace that flow.
  - without good logging, monitoring, and tracing, microservices can become a **black box** nightmare.
  - need for observability

Service Oriented Architecture (SOA)



About SOA

History

- Emerged in early 2000s (though earlier precursors existed) as a response to tightly coupled enterprise systems
- Organizations like large banks, telecoms, and government agencies had sprawling “spaghetti architectures” with multiple legacy systems.
- SOA proposed exposing business functionality as “services” accessible over a network with well-defined contracts.
- Over time, **enterprise vendors** such as IBM, Oracle, SAP promoted SOA solutions, typically involving standards like **WSDL** (Web Services Description Language) and **SOAP** (Simple Object Access Protocol).
- 2005–2010: Big consultancies (Accenture, Deloitte) pushing large SOA transformations.
- Some successes (e.g., major insurance and banking systems). But also large “SOA initiatives” that were over-budget or underdelivered

Goals

reducing integration spaghetti, enabling reusability across enterprise apps, and bringing structure to large distributed environments.

Core idea

Decouple large systems into services communicating via well-defined interfaces.

Each service has a formal interface/contract describing available operations and data formats. Typically, WSDL for SOAP-based services, though the principle is contract-first design.

Services register in a directory (like **UDDI** in early SOA) for clients to find them.

Typical SOA Stack

1. **Enterprise Service Bus (ESB):** A central “bus” product that routes messages, applies transformations, handles security policies, and orchestrates service interactions. Products included Mule ESB, IBM WebSphere ESB, Oracle Service Bus, etc.
2. **SOAP/Web Services:** Heavily used for synchronous calls. Sometimes JMS or other messaging for async. SOAP was verbose XML-based and used WSDL for interface definitions.
3. **BPEL/BPM:** For orchestrating multiple services into complex workflows (Business Process Execution Language). This was big in enterprise contexts (e.g., for an insurance claims process spanning multiple systems).
4. **Governance Tools:** Central registries for service discovery, policy enforcement, versioning, lifecycle management (often expensive, enterprise-oriented software).
5. **Security Standards:** WS-Security, SAML tokens for auth, etc.

Keywords: *WSDL, SOAP, ESB, WS- Specifications* (WS-Security, WS-Addressing, WS-Policy), Message-Oriented Middleware (MOM), BPEL (Business Process Execution Language), Canonical Data Model.*

How Microservices Differ

1. **Granularity**
    - Microservices are smaller, focusing on a single bounded context.
    - Traditional SOA services might be large “Customer Service” or “Billing Service” with many operations.

2. **Decentralized vs Centralized**
    - Microservices prefer decentralized governance (each team decides how to build/deploy).
    - Classic SOA often had a central ESB and a central architecture board.
3. **Data Ownership**
    - Microservices strongly advocate *database-per-service*.
    - SOA projects often used a big enterprise data model (shared).
4. **Technology Stacks**
    - Microservices typically use lightweight protocols (REST/JSON, gRPC), container orchestration, domain-driven design.
    - SOA often used SOAP, ESB, or BPM tooling.
5. **DevOps & CI/CD**
    - Microservices emerged in the era of DevOps, cloud, continuous delivery.
    - SOA often preceded this cultural shift and was more ops-segregated.
6. **ESB vs "Smart Endpoints and Dumb Pipes"**
    - Microservices principle: keep endpoints (services) "smart" (logic, transformations) and use "dumb pipes" (simple transports like HTTP / Message broker) – avoiding a heavy ESB that can become a choke point.
    - In classic SOA, the ESB might orchestrate entire workflows, transforming and routing requests. Microservices prefer distributed coordination (choreography or sagas) with each service containing its own logic.
7. **Synchronous vs Asynchronous**
    - In SOA, synchronous calls with SOAP were very common. Asynchronous messaging also existed, especially in big enterprises, typically with JMS or IBM MQ.
    - Microservices also use both. Synchronous REST/gRPC is popular for direct queries, but asynchronous event-based communication is more flexible for **decoupled** workflows (enabling eventual consistency). Many microservices architectures incorporate an **event streaming platform** like Kafka for high-throughput data streaming and decoupling producers/consumers. This approach might be viewed as an evolution of older message brokers in SOA.

Service Oriented Ambiguity — the term "Service Oriented Architecture" is often used in a lot of different contexts with different meanings.

Doubts

Usecases of Proxy servers?

1. Hide your IP address – better privacy, prevents companies from tracking your activity
    - You want to book a flight from Mumbai to Delhi

- Day 1: go to google, & search for flights.. open all the various websites that you can find. But don't book. Just note the prices
- Day 2: do the search again, all the prices will have gone up significantly (20-50%)
- workaround
    i. If you try from a friend's laptop (different IP address) the prices will still be low
    ii. try to clear your browser cache
        1. will work some of the times
        2. but not always
    iii. you can use a proxy server

2. Bypassing restrictions
    - Tiktok is banned in India, but you can access it via a proxy server
3. Centralized caching
    - if someone inside a college hostel watches a engaging video, it will quickly get viral inside the hostel
    - everyone will try to watch (download) the same video
    - College had setup a simple proxy server
        i. IITB
        ii. we've also set this up for Scaler School of Technology
    - any videos that are being watched are only downloaded once & then they're cached!
    - this drastically reduces the amount of network egress the college has to pay for!

(most) resources online about LLD/HLD are incorrect

LLD - Builder Pattern

```
class User {
    private String name;
    private String address;
    ...

    private User(String name, String address) {
        this.name = name;
        this.address = address;
    }


    public class Builder {
        private String name;
        private String address;
```

```
        public Builder setName(String name) {
            this.name = name;
            return this;
        }
    }
}
```

~~User u = new User("Ivan", "Milkyway");~~

```
User u = new User.Builder()
              .setAddress("Milkyway, Local supercluster")
              .setName("Ivan")
              .build();
```

Q: Can you write this builder pattern in python?

All the people that answered yes do NOT understand the builder pattern!

What does the Builder pattern do?

1. Java doesn't allow you to change the order of function arguments
    a. if you use builder pattern, you can!
2. Java doesn't have named parameters? (name=Ivan, address=Milkyway)
    a. if you use builder pattern, you can!
3. Java doesn't have optional parameters
    a. you can do function overloading, but that requires 2^n combinations!
    b. builder pattern allows you make any and all parameters optional
4. other goodies
    a. validation
    b. if you need to "prepare" some resource before you create the object — delayed instantiation

Python/Ruby/JS

1. change the order
2. named params
3. optional params

all of these are builtin features!

There's no point of using the builder pattern in these languages!

In java, people tell you to pre-emptively create getters & setters. Do you know why?

Bad

```
class User {
   public String name;
}
class Main {
    public static void main() {
        User u = new User();
        u.name = “Haha I can change it!”;
    }
}
```

Good

```
class User {
   private String name;
   public String getName() { return name; }
   public void setName(String name) { this.name = name; }
}

class Main {
    public static void main() {
        User u = new User();
        u.setName(“Haha I can change it!”);
        // I didn’t really protect anything!
    }
}
```

Incorrect reasons

1. Restrict data access
2. security leak
3. data validation
4. reflections
5. separation of concerns
6. abstraction

Correct reason

1. to ensure backwards compatibility!

a. even though you're not doing any validation right now, and there's absolutely no need for the getter & setter
b. tomorrow you might add validation
c. and that will break the client's existing code
d. so to ensure that the client's code doesn't break later on, you force the client to access via the getters & setters from day 1

```
class User:
   name: str


u = User()
u.name = "haha, I can change it!"
```

In python there's no backwards compatibility issue!

Tomorrow if you have to add validation

```
class User:
    _name: str

   @property
    def name(self):
        return self._name

    @name.setter
    def set_name(self, name: str):
        self._name = name


u = User()
u.name = "haha, I can change it!"
```

Why HLD blog posts that discuss the architecture of the entire company are useless

CTO

- will you be responsible for the entire architecture? Not in one go.. you built it step by step, on day 1 you had a ver simple architecture

Senior Architect
- you will not be responsible for the entire architecture
- responsible end-to-end for one large product vertical

Senior Developer
- you will be working on 1 project at a time

Interview
- you will tackling one system at a time

SideCar pattern

How Atlassian reduced latency by 70% by using Sidecar pattern and when should you …

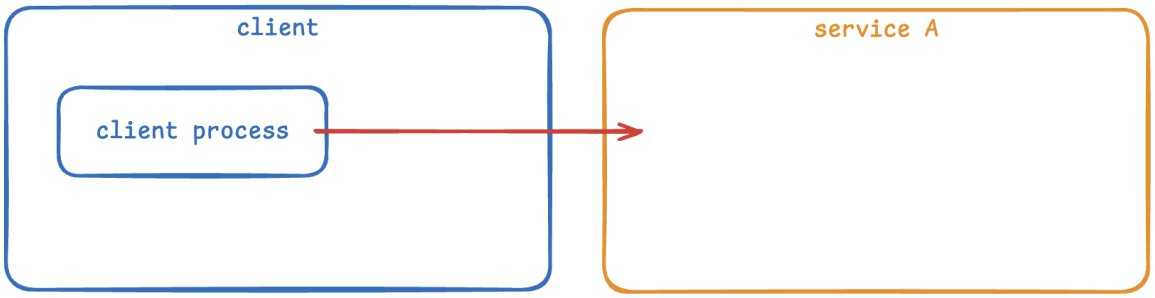
Team A: you have exposed some API that other teams (microservices) can call.
- Imagine that Team A has put some rate limiters on their API, and have laid out detailed guidelines on how to best use this API

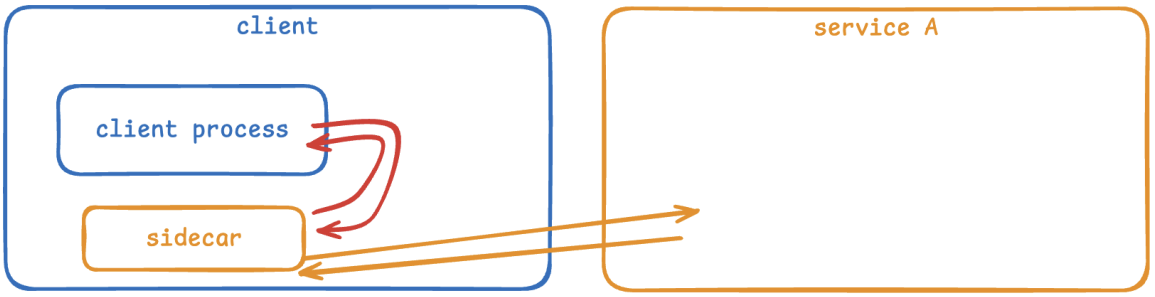
Unfortunately, irrespective of how detailed your guidelines are, other teams might not necessarily follow them.

If team A wants to
1. enforce these guidelines
2. assist the other teams into following these guidelines

they can provide a “sidecar”

A sidecar can be
1. separate process that runs on the client server
2. SDK or library that the client can use to call the API


client
client process
service A


client
client process
sidecar
service A